# Advanced KQL for SecOps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

# Contents


Executive Summary — Guide: Advanced KQL for SecOps 4
- The problem . . . 4
- The solution . . . 4
- How it works . . . 4
- Why it matters . . . 5
- Important boundaries . . . 5

Solution Overview — Guide: Advanced KQL for SecOps 5
- What this is . . . 5
- How the technology works (the mental model) . . . 5
- The standard deployment model . . . 6
- Scope — what this guide covers . . . 7
- Out of scope (handled elsewhere) . . . 7
- Authentication and connections used . . . 7

Requirements — Guide: Advanced KQL for SecOps 8
- R1. Advanced hunting access in Defender XDR . . . 8
- R2. Telemetry actually flowing (tables have data) . . . 8
- R3. A Microsoft Sentinel workspace (for the cross-tool labs) . . . 9
- R4. A learner cohort and lab roles (least privilege) . . . 9
- R5. A safe, read-only lab posture . . . 9
- R6. The PowerShell helper toolchain . . . 10

Global Requirements — Defender (category-level) 10
- G1. Microsoft 365 / Entra tenant . . . 10
- G2. Defender licensing . . . 10
- G3. Administrative roles (least privilege) . . . 11
- G4. Onboarding reach to endpoints / identity / workloads . . . 11
- G5. Network access to Defender service endpoints . . . 11
- G6. A pilot ring . . . 11
- G7. Log Analytics workspace (where required) . . . 11
- G8. Response governance . . . 11

Implementation — Guide: Advanced KQL for SecOps (Guided Labs) 12
- Phase 0 — Assess (read-only, changes nothing) . . . 12
- Phase 1 — Configure (load the lab pack; nothing production yet) . . . 13
- Phase 2 — Pilot (the guided labs) . . . 13
- Phase 3 — Validate (capstone + knowledge-check) . . . 15
- Phase 4 — Expand (operationalize the skills) . . . 15

Rollback / safe-off . . . 15
Testing & Validation — Guide: Advanced KQL for SecOps 16
Gate 1 — Advanced hunting is reachable and tables have data . . . 16
Gate 2 — The Sentinel cross-tool labs can run (where in scope) . . . 16
Gate 3 — Each lab query resolves and returns explainable results . . . 16
Gate 4 — Learners can write joins and aggregations unaided . . . 17
Gate 5 — The lab detection exists and is alert-only . . . 17
Gate 6 — The saved-query set and workbook exist . . . 17
Gate 7 — The knowledge-check passes . . . 17
Gate 8 — Read-only posture held; roles least-privilege . . . 17
Troubleshooting — Guide: Advanced KQL for SecOps 18
1. A lab query returns nothing (empty result) . . . 18
2. "Unknown table" or "unknown column" error . . . 18
3. A `join` returns far too many (or too few) rows . . . 19
4. The same hunt behaves differently in Defender vs Sentinel . . . 19
5. `WSKQL-RunLab.ps1` fails to run a lab query . . . 19
6. "Create detection rule" is greyed out . . . 19
7. The custom detection is firing constantly (noisy) . . . 19
8. Sentinel labs can’t run — no workspace . . . 20
9. Graph connection or permission errors . . . 20
10. Knowledge-check score looks wrong . . . 20
Best Practices — Guide: Advanced KQL for SecOps 20
Write KQL that a teammate can read . . . 20
Hunt on a question, not a hunch . . . 21
Know your two surfaces . . . 21
Turn hunts into durable coverage — carefully . . . 21
Run it as a program, not a one-off class . . . 21
Where to go next . . . 22
Deliverables Summary — Guide: Advanced KQL for SecOps 22
Documentation . . . 22
Lab pack + PowerShell helpers (5) . . . 23
Named objects (verbatim — used across docs, scripts, and tests) . . . 24
What this guide deliberately does NOT do . . . 24
How it is delivered . . . 25
Appendix A — PowerShell Scripts 26

# Executive Summary — Guide: Advanced KQL for SecOps

## The problem

Every modern Microsoft security stack — Defender XDR, Defender for Endpoint, Defender for Office 365, and Microsoft Sentinel — runs on the same query language: **KQL (Kusto Query Language)**. It is the difference between a SOC that can only read the alerts the vendor pre-built and a SOC that can **ask its own questions of its own data**. Most teams that own these tools use them at a fraction of their value, because the skill to drive them was never built:

- **Alert-limited hunting.** Analysts can triage what fires, but cannot pivot — "who else touched this host?", "what did this identity do in the hour before the alert?" — because they cannot write the join.
- **Copy-paste queries no one understands.** Teams paste hunting queries from the internet, cannot adapt them to their own tables, and cannot tell a good result from a false one.
- **No path from a hunt to a durable detection.** Even when someone finds something, the knowledge evaporates — it never becomes a saved query, a custom detection, or a workbook the whole team reuses.

## The solution

This is a **structured enablement guide** — concepts, **guided hands-on labs**, a **take-home runbook**, and a **knowledge-check** — that takes an admin or analyst from KQL fundamentals to **advanced joins, aggregations, cross-table hunting, detection authoring, and workbooks** across **Defender XDR advanced hunting and Microsoft Sentinel**. It is deliberately a **skills** product:

- **Teaches the language, not just the clicks** — the mental model of tables, operators, the tabular pipeline, and how the same KQL runs in both Defender advanced hunting and Sentinel logs.
- **Ships a real, runnable KQL lab pack** — a progressive query library (`WSKQL-Labs.kql`) learners execute in a safe, read-only sandbox and keep afterward as a starting library.
- **Ends in durable artifacts** — a shared saved-query set, an alert-only custom detection, and a workbook the learner builds and the team keeps.
- **Includes a knowledge-check** — so you can prove the skills landed, not just that the class happened.

## How it works

We follow the same repeatable model every solution in this program uses — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — reframed for enablement. We first confirm the learners and the tenant are ready (advanced-hunting access, a Sentinel workspace where used), stand up a small, safe **lab sandbox** and a learner **cohort group**, run the **guided labs** against real telemetry read-only, **validate** the skills with a capstone and a knowledge-check, and then **expand** — folding the new skills, saved queries, and detection into the live hunting and detection program.

The package ships as **self-guided, click-by-click documentation plus a runnable KQL lab pack and four PowerShell helpers** that check readiness, run a chosen lab query read-only, score the

knowledge-check, and report cohort progress — all **read-only**, with **no credentials in code**.

### Why it matters

• **The team can ask its own questions** — pivot across endpoint, identity, email, and cloud tables instead of waiting for a pre-built alert.

• **Hunts become detections** — the guide closes the loop from "I found something once" to a saved query, a custom detection, and a workbook the whole team reuses.

• **One language, every tool** — the same KQL skill pays off in Defender XDR *and* Sentinel, so the investment compounds across the stack.

• **A fraction of consulting cost** — the same field-tested method a Defender/Sentinel specialist would teach, delivered as a repeatable, tested runbook you keep and re-run for new hires.

### Important boundaries

This is a **skills / enablement** product — it teaches people to hunt and build detections. It is **adjacent to, and deliberately does not duplicate**, the catalog's **Threat Hunting hunt-library** solution (a curated, ongoing *hunt program* with a maintained query set and cadence) — this guide builds the **capability**; that product supplies and operates a **library**. It **cross-references rather than repeats** the Defender XDR onboarding, Sentinel analytics/SOAR, and per-pillar (Endpoint / Office 365 / Identity / Cloud Apps) solutions, which stand up the platforms this guide queries. It provides the **technical skill and the labs**, not the **decisions**: which detections to promote to production, what counts as an acceptable false-positive rate, and every production change are the customer's **security-operations calls**. KQL table and column names, and which features are in preview or need a specific plan (e.g. Sentinel, Defender for Endpoint P2), **evolve and can differ by cloud (Commercial vs GCC / GCC High)** — *verify current availability and licensing in your tenant*.

## Solution Overview — Guide: Advanced KQL for SecOps

### What this is

A complete, self-guided **enablement guide** that takes an admin or SOC analyst from **KQL fundamentals** to **advanced joins, aggregations, cross-table threat hunting, custom-detection authoring, and workbook building** across **Microsoft Defender XDR advanced hunting** and **Microsoft Sentinel**. It is delivered as concepts, **guided hands-on labs**, a **runnable KQL lab pack**, a **take-home runbook**, and a **knowledge-check**.

It is a **skills** product — the outcome is people who can drive the tools. It is **adjacent to** the catalog's **Threat Hunting hunt-library** solution: that product delivers and operates a curated, maintained *hunt program*; this guide builds the *capability* to hunt and to author detections. The two are complementary and are cross-referenced, not duplicated.

### How the technology works (the mental model)

• **KQL is a tabular pipeline.** A query starts from a **table** (a data source), then passes rows

through a sequence of **operators** joined by the pipe `|` — each operator takes a table in and emits a table out. Master a dozen operators (`where`, `project`, `extend`, `summarize`, `join`, `sort`, `top`, `count`, `parse`, `mv-expand`) and you can express almost any hunt.

• **Two query surfaces, one language.** The **Defender advanced hunting** surface

(`security.microsoft.com → Hunting → Advanced hunting`) queries the Defender XDR schema — `DeviceProcessEvents`, `DeviceNetworkEvents`, `IdentityLogonEvents`, `EmailEvents`, `AlertInfo`, `AlertEvidence`, and more. **Microsoft Sentinel** (in the Azure/Defender portal) queries a **Log Analytics workspace** — `SecurityEvent`, `SigninLogs`, `AuditLogs`, `CommonSecurityLog`, plus any connected data. The **same KQL** runs in both; the **tables differ**. *Table and column names evolve and can differ by cloud — verify against your tenant's schema.*

• **Joins are how you correlate.** Real hunts span tables — "processes on a host that later made a

suspicious network connection", "the sign-in that preceded a mailbox rule change". `join kind=inner / leftouter / leftanti` on a shared key (device id, account, `NetworkMessageId`, time-window) is the core hunting skill this guide builds.

• **Aggregations turn events into signal.** `summarize` with `count()`, `dcount()`, `make_set()`,

`arg_max()`, and `bin()` collapses raw events into rates, first-seen/last-seen, and per-entity rollups — the shape most detections and workbooks are built on.

• **From hunt to detection.** A hunting query that reliably isolates malicious behavior becomes a

**custom detection rule** (Defender) or an **analytics rule** (Sentinel) that generates an alert on a schedule — the loop that turns a one-time find into durable coverage.

• **Workbooks visualize.** A saved query bound to a **workbook** tile (time chart, table, tile)

turns a hunt into a monitorable, shareable view for the team.

## The standard deployment model

Every solution in this program follows the same five stages; here they structure the guide:

1. **Assess.** Confirm the learners and tenant are ready — advanced-hunting access, a Sentinel

workspace where used, and that the tables have data (run `WSKQL-Discovery.ps1`).

1. **Configure.** Create the learner cohort group `Grp-KQLGuide-Cohort`; agree a **safe,

read-only lab posture**; load the KQL lab pack `WSKQL-Labs.kql` and save the starter shared query set `Hunt-KQLLab-Pilot`.

1. **Pilot.** Run the **guided labs** (Labs 1–8) with the cohort against real telemetry, **read-only**

— fundamentals → filtering → aggregation → joins → cross-table hunting → detection → workbook.

1. **Validate.** Run the **capstone** hunt and score the **knowledge-check** `Check-KQLLab-Pilot`

(`WSKQL-Validate.ps1`), and confirm the built detection `Detect-KQLLab-Pilot` is **alert-only**.

1. **Expand.** Fold the skills, the saved query set, the alert-only detection, and the workbook

`Workbook-KQLLab-Pilot` into the **live hunting and detection program** — and decide, as an operations decision, which lab detections to promote to production.

## Scope — what this guide covers

- **KQL fundamentals** — tables, the tabular pipeline, `where` / `project` / `extend` / `sort` / `take` / `count`, and reading the schema.
- **Aggregations** — `summarize`, `count` / `dcount` / `make_set` / `arg_max`, `bin()` time-bucketing.
- **Joins & correlation** — `join` kinds across Defender XDR tables and across Sentinel tables.
- **Cross-tool hunting patterns** — the same hunt expressed in Defender advanced hunting and in Sentinel; when to use which surface.
- **Detection authoring** — turning a tuned hunt into an **alert-only** custom detection / analytics rule.
- **Workbooks** — binding a query to a workbook tile for a monitorable view.
- **A take-home runbook + knowledge-check** — so the skills persist and can be proven.

## Out of scope (handled elsewhere)

- **Standing up the platforms.** Onboarding Defender XDR, connecting data to Sentinel, and licensing the pillars are the **Defender XDR onboarding**, **Sentinel deployment**, and per-pillar catalog solutions. This guide **queries** those platforms; it does not deploy them.
- **A maintained, operated hunt library / program.** The **Threat Hunting hunt-library** solution delivers and runs a curated hunt program on a cadence. This guide teaches the skill and ships a **learning** lab pack — it is not a replacement for that ongoing library.
- **SOAR / automated response.** Building Sentinel playbooks and automated response is the Sentinel SOAR catalog area; the labs stop at **alert-only** detections.
- **The security-operations decisions.** Which detections to promote, acceptable false-positive rates, and every production change are **the customer's SOC / security-leadership determinations**; this guide provides the skill, the labs, and the data.

## Authentication and connections used

- **Microsoft Defender portal** (`security.microsoft.com`) for **Advanced hunting**, custom detection rules, and (where integrated) Sentinel.
- **Microsoft Sentinel** on a **Log Analytics workspace** for KQL over connected logs, analytics rules, and workbooks.
- **PowerShell** with **Microsoft.Graph** (security — advanced hunting via the

`security/runHuntingQuery` endpoint) and, where the Sentinel/Log Analytics labs are run programmatically, the **Az** modules — used only to **run lab queries read-only**, check readiness, score the knowledge-check, and report progress.

- All helpers authenticate **interactively** with an MFA-capable account holding at least read/hunt

scopes; **no credentials are stored in code**. Government-cloud tenants pass the matching endpoints — *verify endpoints for your cloud*.

# Requirements — Guide: Advanced KQL for SecOps

These are the **solution-specific** prerequisites for running the KQL enablement guide. The **category baseline** — an active Microsoft 365 / Entra tenant, Defender licensing in general, least-privilege admin roles, onboarding reach, network access to Defender endpoints, a pilot ring, a Log Analytics workspace where required, and response governance — is covered once in `global-requirements-defender.md` (**G1–G8**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

## R1. Advanced hunting access in Defender XDR

- **Description:** The learners can open **`security.microsoft.com → Investigation & response →

Hunting → Advanced hunting`** and run queries against the Defender XDR schema.

- **Why required:** The core labs run KQL against the Defender advanced-hunting tables. Without

access there is nothing to query.

- **How to verify:** Sign in and confirm the **Advanced hunting** blade loads and a trivial query

(`DeviceInfo | take 1`) returns. Or run `WSKQL-Discovery.ps1`, which checks table reachability and freshness.

- **How to implement if missing:** Grant a Defender role that includes advanced hunting (e.g.

**Security Reader** plus the hunting permission, or **Security Operator**); confirm the pillars are onboarded so tables have data (that onboarding is the per-pillar catalog solutions, not this one).

## R2. Telemetry actually flowing (tables have data)

- **Description:** The Defender XDR tables in scope (`DeviceProcessEvents`, `DeviceNetworkEvents`,

`IdentityLogonEvents`, `EmailEvents`, `AlertEvidence`, …) — and, for the Sentinel labs, the workspace tables (`SigninLogs`, `SecurityEvent`, …) — return recent rows.

- **Why required:** Labs teach hunting on **real** data. Empty tables make the joins and aggregations

meaningless and hide schema mistakes.

- **How to verify:** In Advanced hunting run `DeviceProcessEvents | where Timestamp > ago(24h) |

count; in Sentinel run SigninLogs | where TimeGenerated > ago(24h) | count. Both return rows. Or read the freshness section of WSKQL-Discovery.ps1`.

• **How to implement if missing:** Confirm the relevant pillars are onboarded and licensed (the per-pillar solutions); allow ingestion time. Labs can still teach syntax on whatever tables do have data — the discovery script reports which are live.

## R3. A Microsoft Sentinel workspace (for the cross-tool labs)

• **Description:** A **Microsoft Sentinel**-enabled **Log Analytics workspace** with at least one connected data source, for the labs that contrast Defender advanced hunting with Sentinel.

• **Why required:** Part of the guide is teaching that the **same KQL** runs against **different tables** in the two surfaces, and when to use which. *Sentinel is a licensed/metered Azure service — verify current availability and cost model in your tenant.*

• **How to verify:** Open Sentinel, confirm the workspace loads and **Logs** runs a query; confirm at least one connector is ingesting (e.g. Entra sign-in logs).

• **How to implement if missing:** If no Sentinel workspace exists, the Defender-only labs still run in full; the Sentinel labs can be deferred or run in a demo/eval workspace. Standing up Sentinel is the **Sentinel deployment** catalog solution, not this guide.

## R4. A learner cohort and lab roles (least privilege)

• **Description:** A named group of learners — `Grp-KQLGuide-Cohort` — each holding a **read / hunt role (e.g.** Security Reader **with hunting, or** Security Operator**). Authoring the lab** custom detection **needs a role that can create detection rules (**Security Administrator** or the granular equivalent), used by the instructor.

• **Why required:** Labs should run at **least privilege** — reading and hunting, not changing tenant security posture. Only the capstone detection-authoring step needs elevation, and only for the instructor.

• **How to verify:** Entra → **Roles and administrators** (or Defender **Permissions & roles**) shows the cohort holds read/hunt roles and the instructor holds the authoring role.

• **How to implement if missing:** Create `Grp-KQLGuide-Cohort` in Entra; assign read/hunt roles to learners (prefer **PIM-eligible** for any elevated role where Entra ID P2 is available — *verify licensing*). Reserve detection-authoring rights to the instructor.

## R5. A safe, read-only lab posture

• **Description:** An agreed rule that all labs are **read-only** — queries and (at the capstone only) an **alert-only** detection — with **no response actions** (isolate, delete, disable) taken during the guide.

• **Why required:** Learners run against **production telemetry**. Hunting is read-only by nature, but

the posture must be explicit so no one experiments with a response action on a live host or mailbox.

• **How to verify:** Confirm the guide plan states read-only; confirm the capstone detection is created **alert-only** (no automatic remediation).

• **How to implement if missing:** State it in the guide kickoff; keep response actions out of scope and route any real finding to the SOC's normal incident process (category **G8**).

### R6. The PowerShell helper toolchain

• **Description:** A workstation with **Microsoft.Graph** (security modules) and, for the Sentinel labs run programmatically, the **Az** modules, signed in with an MFA-capable account holding at least read/hunt scopes; plus access to **Advanced hunting** in the portal for the interactive labs.

• **Why required:** The four delivered helpers check readiness, run a chosen lab query read-only, score the knowledge-check, and report cohort progress. The KQL lab pack itself runs in the portal or via the helper.

• **How to verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a version; `Connect-MgGraph` succeeds; Advanced hunting loads in the portal.

• **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser` (and `Az` if running the Sentinel labs from PowerShell); sign in. For Government cloud, connect with the matching environment/endpoints — *verify the correct endpoints for your cloud*.

## Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

### G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

### G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why:

Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

## G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

## G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

## G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

## G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

## G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

## G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

# Implementation — Guide: Advanced KQL for SecOps (Guided Labs)

This is the exact, ordered guide — a sequence of **guided hands-on labs**. Every object you create is **named** with a consistent convention — `<Purpose> – <Scope> – <Stage>` — and the same name is reused in later labs, the knowledge-check, and the scripts. Run the labs against **real telemetry, read-only**; the only change to the tenant is a single **alert-only** capstone detection.

> ***Before you start:*** *confirm advanced-hunting access (**R1**), that tables have data (**R2**), a Sentinel workspace for the cross-tool labs (**R3**), the cohort and roles (**R4**), and the **read-only** posture (**R5**). This guide **queries** the platforms; it does not deploy or reconfigure them.*

**Named objects created in this guide (verbatim — reused everywhere):**

| Object | Name | What it is |
|---|---|---|
| Learner cohort group | `Grp-KQLGuide-Cohort` | Entra security group of guide learners |
| Shared saved-query set | `Hunt-KQLLab-Pilot` | The lab queries saved to shared queries for reuse |
| Custom detection (capstone) | `Detect-KQLLab-Pilot` | Alert-only detection built from a tuned lab hunt |
| Workbook (lab) | `Workbook-KQLLab-Pilot` | Workbook built in the visualization lab |
| Knowledge-check | `Check-KQLLab-Pilot` | The scored end-of-guide knowledge-check |

The runnable queries for every lab live in the delivered `WSKQL-Labs.kql` pack, organized by lab number. You can paste them into Advanced hunting / Sentinel Logs, or run one read-only via `WSKQL-RunLab.ps1`.

## Phase 0 — Assess (read-only, changes nothing)

### *Step 0.1 — Confirm readiness*

1. Sign in to the **Microsoft Defender portal** → `https://security.microsoft.com` and open

**Investigation & response** → **Hunting** → **Advanced hunting**.

1. Run `DeviceInfo | take 1` — confirm it returns.
2. From PowerShell, inventory readiness across both surfaces:

```
pwsh -File .\WSKQL-Discovery.ps1
```

It reports which Defender XDR tables have recent data, whether a Sentinel workspace is reachable, and the cohort-group membership. **Read-only.**

### *Step 0.2 — Create the learner cohort*

1. In **Entra admin center** (`entra.microsoft.com`) → **Groups** → **New group**.
2. Type **Security**; **Name** `Grp-KQLGuide-Cohort`; add the learners.
3. Assign each learner a **read/hunt** role and reserve detection-authoring to the instructor (**R4**).

## Phase 1 — Configure (load the lab pack; nothing production yet)

### *Step 1.1 — Load the KQL lab pack*

1. Open `WSKQL-Labs.kql` (delivered). It is organized into labeled sections — **Lab 1 … Lab 8** plus the **Capstone**.

1. Confirm you can paste a query into **Advanced hunting** and run it, and (for the Sentinel labs) into **Sentinel** → **Logs**.

### *Step 1.2 — Create the shared saved-query set*

1. In Advanced hunting, save your first working query into **shared queries** under a folder named `Hunt-KQLLab-Pilot`. As each lab produces a good query, save it here so the cohort reuses one set. *Verify current shared-query behavior in your tenant.*

## Phase 2 — Pilot (the guided labs)

Run each lab with the cohort. Each lab states the **concept**, the **task**, and the **check**.

### *Lab 1 — Fundamentals: tables and the tabular pipeline*

• **Concept.** A query starts from a table and pipes rows through operators. Meet `take`, `where`, `project`, `sort`, `count`.

• **Task.** Run: `DeviceProcessEvents | where Timestamp > ago(24h) | project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine | sort by Timestamp desc | take 20`.

• **Check.** You can explain what each `|` stage does and reorder them.

### *Lab 2 — Filtering and string operators*

• **Concept.** `where` with ==, =~ (case-insensitive), `has`, `contains`, `startswith`, `in`, and `matches regex`. Filter early; filter cheaply (`has` before `contains`).

• **Task.** Find PowerShell executions with an encoded command:

```
DeviceProcessEvents | where FileName =~ "powershell.exe" | where ProcessCommandLine has "-enc"
```

• **Check.** You can narrow a noisy table to a specific behavior and read why `has` beats `contains`.

### *Lab 3 — Aggregations with `summarize`*

• **Concept.** `summarize` collapses rows into per-group signal — `count()`, `dcount()`,

`make_set()`, `arg_max()`, and time-bucketing with `bin()`.

- **Task.** Per device, count distinct destination IPs in the last day:

```
DeviceNetworkEvents | where Timestamp > ago(1d) | summarize dcount(RemoteIP) by DeviceName | sort by dcount_RemoteIP desc
```

- **Task.** Trend logons by hour: `IdentityLogonEvents | summarize count() by bin(Timestamp, 1h)`.
- **Check.** You can turn raw events into a rate and a per-entity rollup.

### *Lab 4 — Joins: correlating two tables*

- **Concept.** `join kind=inner / leftouter / leftanti` on a shared key. Join a **small, filtered** left side to a larger right side.
- **Task.** Find processes on a host that then made an external network connection — join `DeviceProcessEvents` to `DeviceNetworkEvents` on `DeviceId` within a time window.
- **Check.** You can pick the right `join kind` and explain `leftanti` ("in A but not B") as a hunting tool (e.g. expected-but-missing).

### *Lab 5 — Cross-tool hunting: Defender advanced hunting vs Sentinel*

- **Concept.** The **same KQL** runs in both surfaces; the **tables differ**. Defender: `IdentityLogonEvents`, `EmailEvents`. Sentinel: `SigninLogs`, `SecurityEvent`.
- **Task.** Express "risky sign-ins in the last day" first in Defender advanced hunting, then in Sentinel against `SigninLogs`, and note the schema differences.
- **Check.** You can choose the right surface for a question and translate a hunt between schemas.

*Table/column names differ and evolve — verify against your tenant.*

### *Lab 6 — A real hunting pattern: suspicious sequence*

- **Concept.** Chain filters, joins, and aggregation into a hunt that isolates a behavior (e.g. new-process → outbound connection → credential-access indicators).
- **Task.** Build the capstone-candidate query from the pack's **Lab 6** section and tune it until it returns a **small, explainable** result set.
- **Check.** Every row in the result is one you can justify — the hunt is specific, not noisy.

### *Lab 7 — From hunt to detection (alert-only)*

- **Concept.** A tuned hunt becomes a **custom detection rule** that runs on a schedule and generates an **alert only** — no automatic response.
- **Task.** In Advanced hunting, open your Lab 6 query and choose **Create detection rule**. Name it `Detect-KQLLab-Pilot`, set the frequency and impacted entities, and set the response action to **generate an alert only**. *Do not enable any automatic remediation.*
- **Check.** `Detect-KQLLab-Pilot` exists and is alert-only.

### *Lab 8 — Visualize: build a workbook*

• **Concept.** A saved query bound to a workbook tile becomes a monitorable, shareable view.

• **Task.** In Sentinel (or a Defender report surface) create a workbook `Workbook-KQLLab-Pilot` with a time-chart of your Lab 3 aggregation and a table of your Lab 6 hunt. *Verify current workbook location/behavior in your tenant.*

• **Check.** The workbook renders both tiles from live data.

## Phase 3 — Validate (capstone + knowledge-check)

1. **Capstone hunt.** Each learner reproduces the Lab 6 hunt from a blank query editor, unaided, and explains the result.

1. **Knowledge-check.** Score the knowledge-check `Check-KQLLab-Pilot`:

```
pwsh -File .\WSKQL-Validate.ps1
```

It walks the pass/fail gates in `05-testing.md` — readiness, that the lab queries resolve, that `Detect-KQLLab-Pilot` is alert-only, and that the saved-query set and workbook exist — and prints a score.

1. **Cohort report.** Summarize progress for the group:

```
pwsh -File .\WSKQL-Report.ps1 -OutputCsv .\wskql-cohort.csv
```

## Phase 4 — Expand (operationalize the skills)

1. **Adopt the query set.** Move the tuned queries from `Hunt-KQLLab-Pilot` into the team's standing shared-query library so hunts are reused, not reinvented.

1. **Promote detections deliberately.** If `Detect-KQLLab-Pilot` proves quiet and accurate, security operations may **document a decision** to promote it (renamed for production) — an **operations decision**, not a default this guide flips.

1. **Feed the hunt program.** Hand the good queries to the **Threat Hunting hunt-library** solution so they join the maintained, operated program — the guide builds skill; that product runs the library.

1. **Keep learning.** Schedule a follow-up on advanced topics (functions, `materialize`, `let` statements, parsing, Sentinel analytics-rule tuning) and re-run the guide for new hires.

## Rollback / safe-off

• **Queries / labs:** read-only — nothing to roll back.

• **Custom detection:** disable or delete `Detect-KQLLab-Pilot` to stop it generating alerts; it is alert-only and never remediates.

• **Workbook / saved queries / cohort group:** delete `Workbook-KQLLab-Pilot`, the

`Hunt-KQLLab-Pilot` folder, and `Grp-KQLGuide-Cohort` if the guide was one-off; keep them if you are operationalizing.

# Testing & Validation — Guide: Advanced KQL for SecOps

Validate the guide against explicit **pass/fail gates** — both that the **environment** supports the labs and that the **skills** landed. Run the scripts and walk the manual checks. The automated gates are produced by `WSKQL-Validate.ps1`; readiness comes from `WSKQL-Discovery.ps1`, running a lab query from `WSKQL-RunLab.ps1`, and cohort evidence from `WSKQL-Report.ps1`.

Each gate lists **What good looks like / How to test / If it fails**.

## Gate 1 — Advanced hunting is reachable and tables have data

- **What good looks like:** Advanced hunting loads; core Defender XDR tables return recent rows.
- **How to test:** Run `WSKQL-Discovery.ps1`; confirm the freshness section shows `DeviceProcessEvents`,

`DeviceNetworkEvents`, `IdentityLogonEvents` (and others in scope) with rows in the last 24h.

- **If it fails:** Confirm the relevant pillars are onboarded/licensed (per-pillar solutions) and

allow ingestion time; run labs on whatever tables are live meanwhile (**R2**).

## Gate 2 — The Sentinel cross-tool labs can run (where in scope)

- **What good looks like:** A Sentinel workspace is reachable and `SigninLogs` (or another connected

table) returns rows.

- **How to test:** In **Sentinel** → **Logs** run `SigninLogs | where TimeGenerated > ago(24h) | count`;

`WSKQL-Discovery.ps1` reports the workspace as reachable.

- **If it fails:** Run the Defender-only labs in full and defer Lab 5's Sentinel half, or use a demo

workspace; standing up Sentinel is a separate solution (**R3**).

## Gate 3 — Each lab query resolves and returns explainable results

- **What good looks like:** Labs 1–6 from `WSKQL-Labs.kql` run without schema errors and return

results the learner can explain.

- **How to test:** Run a representative lab query read-only:

`pwsh -File .\WSKQL-RunLab.ps1 -Lab 6`. It executes the pack's Lab 6 query via Graph and returns rows. Repeat for other labs as needed.

- **If it fails:** Correct any table/column name to your tenant's current schema (the pack uses

representative names that evolve); re-run.

## Gate 4 — Learners can write joins and aggregations unaided

• **What good looks like:** In the capstone, learners reproduce the Lab 6 hunt from a blank editor — a filtered `where`, a `summarize`, and a `join` — and explain each stage.

• **How to test:** Observe the capstone; confirm the produced query matches the intent (not copy-paste) and the result set is small and justified.

• **If it fails:** Re-run Labs 3 and 4 (aggregations, joins) — these are the two skills most learners need reinforced — before repeating the capstone.

## Gate 5 — The lab detection exists and is alert-only

• **What good looks like:** `Detect-KQLLab-Pilot` exists as a custom detection rule that **generates an alert only** — no automatic remediation.

• **How to test: Advanced hunting** → **Custom detection rules** lists it; its response action is alert-only. `WSKQL-Validate.ps1` confirms.

• **If it fails:** Recreate it from the tuned Lab 6 query (Lab 7) and set the response to alert-only. Never wire a lab detection to a response action.

## Gate 6 — The saved-query set and workbook exist

• **What good looks like:** The `Hunt-KQLLab-Pilot` shared-query folder holds the lab queries, and `Workbook-KQLLab-Pilot` renders its tiles from live data.

• **How to test:** Confirm the shared-query folder and the workbook load and populate; `WSKQL-Validate.ps1` checks for their presence where the API allows.

• **If it fails:** Re-save the queries (Step 1.2) and rebuild the workbook (Lab 8).

## Gate 7 — The knowledge-check passes

• **What good looks like:** Each learner passes `Check-KQLLab-Pilot` (operators, join kinds, aggregation functions, choosing the right surface).

• **How to test:** Run `WSKQL-Validate.ps1`; it prints the knowledge-check score per the answer key.

• **If it fails:** Revisit the specific topic a learner missed; the knowledge-check maps each question to the lab that teaches it.

## Gate 8 — Read-only posture held; roles least-privilege

• **What good looks like:** No response action was taken during the guide; learners held read/hunt roles and only the instructor could author the detection.

• **How to test:** Review **Permissions & roles** (R4) and confirm least-privilege; confirm the

guide plan and audit show read-only activity only.

- **If it fails:** Tighten role assignments; reaffirm the read-only posture (**R5**).

### *Validation summary*

| Gate | Check | Pass condition |
|---|---|---|
| 1 | Advanced hunting + data | Core tables return recent rows |
| 2 | Sentinel labs runnable | Workspace reachable; `SigninLogs` returns (where in scope) |
| 3 | Lab queries resolve | Labs 1–6 run without schema errors |
| 4 | Joins/aggregations unaided | Capstone reproduced from a blank editor |
| 5 | Lab detection alert-only | `Detect-KQLLab-Pilot` exists, alert-only |
| 6 | Saved set + workbook | `Hunt-KQLLab-Pilot` + `Workbook-KQLLab-Pilot` present |
| 7 | Knowledge-check passes | `Check-KQLLab-Pilot` scored to threshold |
| 8 | Read-only + least-privilege | No response actions; roles scoped |

All gates pass ⇒ the cohort is enabled; proceed to **Expand** (operationalize the skills). Any fail ⇒ reinforce the relevant lab and re-check.

# Troubleshooting — Guide: Advanced KQL for SecOps

The common failure modes, each as **symptom** → **cause** → **fix**. Most guide issues are empty tables, schema drift, or a join written the wrong way round — not a broken platform.

## 1. A lab query returns nothing (empty result)

- **Symptom:** A lab query runs without error but returns 0 rows.
- **Cause:** The table has no data in the chosen window, the filter is too tight, or the pillar isn't onboarded.
- **Fix:** Widen the time window (`ago(7d)`), relax the filter, and confirm the table is live with `WSKQL-Discovery.ps1`. If the table is genuinely empty, run the lab on a table that has data (**R2**).

## 2. "Unknown table" or "unknown column" error

- **Symptom:** A query fails to resolve a table or column (e.g. `DeviceProcessEvents`, `RemoteIP`).
- **Cause: Schema drift** — the delivered pack uses representative names that Microsoft renames or

reshapes over time, and some tables differ by cloud (GCC/GCC High).

- **Fix:** Open **Advanced hunting** → **schema** (or Sentinel **Logs** → **schema**) and correct the name to your tenant's current schema. The lab pack is a starting point, not a frozen contract. *Verify current schema.*

## 3. A `join` returns far too many (or too few) rows

- **Symptom:** A join explodes to a huge result, or unexpectedly returns nothing.
- **Cause:** Wrong `join kind`, joining on a non-unique key, or the left side isn't filtered first.
- **Fix:** Filter and `project` the **left** side down before joining; pick the right kind (`inner` = both, `leftouter` = keep all left, `leftanti` = in-left-not-right); confirm the join key is the same entity in both tables (e.g. `DeviceId`, `NetworkMessageId`). Re-run Lab 4.

## 4. The same hunt behaves differently in Defender vs Sentinel

- **Symptom:** A query works in Advanced hunting but errors or returns different data in Sentinel (or vice-versa).
- **Cause:** They query **different schemas** — Defender XDR tables (`IdentityLogonEvents`) are not the Sentinel workspace tables (`SigninLogs`), and the timestamp column differs (`Timestamp` vs `TimeGenerated`).
- **Fix:** Use the surface's own table and timestamp column (Lab 5). The **language** is the same; the **tables** are not. *Verify each surface's schema.*

## 5. `WSKQL-RunLab.ps1` fails to run a lab query

- **Symptom:** The helper errors instead of returning rows.
- **Cause:** Not connected, missing the hunting scope, wrong cloud endpoint, or the pack's query uses a table absent in your tenant.
- **Fix:** Confirm `Connect-MgGraph` succeeds with `ThreatHunting.Read.All`; pass the correct `-Environment` for Government cloud; if a specific lab's table is absent, correct the schema in `WSKQL-Labs.kql` and re-run.

## 6. "Create detection rule" is greyed out

- **Symptom:** In Lab 7 you cannot create the custom detection.
- **Cause:** The account lacks the detection-authoring role — this is the one elevated step.
- **Fix:** Have the **instructor** (holding Security Administrator or the granular equivalent) author `Detect-KQLLab-Pilot`; learners run read-only (**R4**). Keep it **alert-only**.

## 7. The custom detection is firing constantly (noisy)

- **Symptom:** `Detect-KQLLab-Pilot` generates a flood of alerts.
- **Cause:** The underlying hunt wasn't tuned to a small, explainable result before it was promoted.
- **Fix:** Disable the detection, tighten the Lab 6 query (more selective filters, aggregation thresholds) until it returns few, justified rows, then recreate the alert-only rule. Never promote a noisy hunt.

### 8. Sentinel labs can't run — no workspace

- **Symptom:** Lab 5's Sentinel half has nowhere to run.
- **Cause:** No Microsoft Sentinel workspace is enabled in the tenant.
- **Fix:** Run the Defender-only labs in full and defer the Sentinel comparison, or use a demo/eval workspace. Standing up Sentinel is a separate catalog solution (**R3**). *Sentinel is a metered service — verify cost/availability.*

### 9. Graph connection or permission errors

- **Symptom:** `Connect-MgGraph` fails, or a call returns 403.
- **Cause:** Missing modules, wrong cloud endpoint, unconsented scopes, or insufficient role.
- **Fix:** `Install-Module Microsoft.Graph` (and `Az` for the Sentinel labs); connect with the correct `-Environment` for Government cloud; consent the read/hunt scopes in each script header; confirm the account holds a read/hunt role (**R4**). *Verify the correct endpoints for your cloud.*

### 10. Knowledge-check score looks wrong

- **Symptom:** `WSKQL-Validate.ps1` reports a score that doesn't match expectations.
- **Cause:** The check is answered against the delivered answer key; a tenant-specific schema change can make a "correct" query look different.
- **Fix:** The knowledge-check tests **concepts** (operators, join kinds, aggregation functions, choosing a surface), not tenant-specific table names. Review the mapped lab for any question a learner missed; adjust the answer key only if your schema genuinely differs.

## Best Practices — Guide: Advanced KQL for SecOps

Keep-it-clean guidance for turning a KQL guide into a durable hunting-and-detection skill that stays sharp over time.

### Write KQL that a teammate can read

- **Filter early, filter cheaply.** Put the most selective `where` first, and prefer `has` over `contains` (token vs substring) — it is faster and clearer. A hunt that returns fast gets re-run.
- `project` **down to what you need.** Trim columns early so results are readable and joins are cheap;

the reader should see the story, not every field.

- **Name your intent with `let` and comments.** Use `let` for reusable time windows and indicator lists, and a one-line comment per query stating the question it answers. Queries you save should be self-explanatory a month later.

## Hunt on a question, not a hunch

- **Start every hunt from an answerable question** — "who else ran this binary?", "what did this identity do before the alert?" — then pivot with joins. Save the good hunts to `Hunt-KQLLab-Pilot` so the team reuses them.
- **Aim for a small, explainable result.** A hunt that returns thousands of rows is not tuned. Push filters and aggregation until every row is one you can justify.
- **Re-hunt after every incident.** A new indicator (hash, IP, account) is a new query — run it across the whole tenant, not just the one reported entity.

## Know your two surfaces

- **Same language, different tables.** Reach for **Defender advanced hunting** for endpoint / identity / email XDR tables; reach for **Sentinel** for connected logs and long-retention correlation. Teach learners to choose the right surface, not memorize one.
- **Verify the schema — it drifts.** Microsoft renames and reshapes hunting tables periodically. Treat every delivered query as a **starting point**, and confirm table/column names against your tenant. *Verify current schema.*

## Turn hunts into durable coverage — carefully

- **Alert-only first.** Promote a hunt to a custom detection (`Detect-KQLLab-Pilot`) only once it is quiet and accurate, and keep it **alert-only** until proven. A noisy detection trains the team to ignore alerts.
- **Promote deliberately.** Moving a lab detection to production, tightening its schedule, or wiring any response is a **documented operations decision** — the SOC's call, made with rollback in mind, not a default this guide flips.
- **Version your queries.** Keep the KQL library in source control so hunts are reviewable and portable — the same discipline the Sentinel detection-as-code solution applies.

## Run it as a program, not a one-off class

- **Practice keeps the skill.** KQL fades without use. Schedule recurring hunt sessions and rotate who writes the query of the week.
- **Feed the hunt library.** Hand tuned queries to the **Threat Hunting hunt-library** solution so they join the maintained, operated program — the guide builds the skill; that product runs the library.

• **Re-run for new hires.** The take-home runbook and knowledge-check make onboarding a new analyst

repeatable.

### Where to go next

With the team able to hunt and author detections, the natural next steps in this catalog are:

• **Threat Hunting hunt-library** — adopt and operate a curated, maintained hunt program built on the skill this guide gives you (adjacent, not duplicate).

• **Microsoft Sentinel analytics rules & SOAR** — turn trusted detections into scheduled analytics rules and automated playbooks beyond alert-only.

• **Microsoft Defender XDR onboarding** — ensure every pillar's tables are present so hunts span the whole kill chain.

• **The per-pillar guides** — **Defender for Endpoint** and **Defender for Office 365** guides apply the same KQL skill to endpoint and email hunting specifically.

## Deliverables Summary — Guide: Advanced KQL for SecOps

Everything included in this guide, and what each piece is for.

### Documentation

| Document | Purpose |
|---|---|
| **Executive Summary** (`executive-summary.md`) | One-page framing: the "alert-limited, can't-write-the-join" problem, the enablement solution (concepts + labs + runbook + knowledge-check), and the boundary (skills product; adjacent to the Threat Hunting hunt-library; queries the platforms, does not deploy them). |
| **Solution Overview** (`01-solution-overview.md`) | What it is, how KQL works (tabular pipeline, the two query surfaces, joins, aggregations, hunt→detection, workbooks), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used. |
| **Requirements** (`02-requirements.md`) | Solution-specific prerequisites (advanced-hunting access, telemetry flowing, a Sentinel workspace for the cross-tool labs, a cohort + least-privilege roles, a read-only posture, the toolchain) — layered on the shared `global-requirements-defender.md` (G1–G8). |

| Document | Purpose |
|---|---|
| **Implementation** (`03-implementation.md`) | The guided labs: assess readiness, create the cohort, load the KQL lab pack, run Labs 1–8 (fundamentals → filtering → aggregation → joins → cross-tool hunting → real hunt → alert-only detection → workbook), then capstone + knowledge-check, then operationalize — every object named, with rollback. |
| **Testing & Validation** (`05-testing.md`) | Eight pass/fail gates covering both environment readiness (tables have data, Sentinel reachable) and skills landed (joins/aggregations unaided, alert-only detection, saved set + workbook, knowledge-check passes), referencing the scripts by name. |
| **Troubleshooting** (`06-troubleshooting.md`) | The common failure modes (empty tables, schema drift, join errors, cross-tool confusion, detection not alert-only, connection/permission errors) as symptom → cause → fix. |
| **Best Practices** (`07-best-practices.md`) | Write-readable-KQL, hunt-on-a-question, know-your-two-surfaces, turn-hunts-into-durable-coverage-carefully, run-it-as-a-program — plus where to go next in the catalog. |

## Lab pack + PowerShell helpers (5)

All helpers are **read-only**, authenticate **interactively** (no credentials in code), and use **Microsoft.Graph** (security — advanced hunting via `runHuntingQuery`) and, for the Sentinel labs, **Az**. Government-cloud tenants pass the matching environment — *verify endpoints for your cloud*.

| File | Type | What it does |
|---|---|---|
| `WSKQL-Labs.kql` | KQL lab pack | The runnable, progressive query library — one labeled section per lab (fundamentals, filtering, aggregation, joins, cross-tool, real hunt, detection candidate, workbook query) plus the capstone. The core take-home artifact. |
| `WSKQL-Discovery.ps1` | Read-only | Inventories readiness: which Defender XDR tables have recent data, whether a Sentinel workspace is reachable, and the cohort-group membership. Changes nothing. |

| File | Type | What it does |
|---|---|---|
| `WSKQL-RunLab.ps1` | Read-only | Runs a chosen lab query (`-Lab <n>`) from the pack via Graph and returns the rows — so learners can execute a lab from PowerShell as well as the portal. |
| `WSKQL-Validate.ps1` | Read-only | Runs the `05-testing.md` gates as PASS/FAIL/MANUAL lines and scores the knowledge-check `Check-KQLLab-Pilot`: readiness, lab queries resolve, detection alert-only, saved set + workbook present. |
| `WSKQL-Report.ps1` | Read-only | Reports cohort progress and lab-result summaries (rows returned per lab, knowledge-check scores); exports to CSV. |

## Named objects (verbatim — used across docs, scripts, and tests)

- **Learner cohort group:** `Grp-KQLGuide-Cohort`
- **Shared saved-query set:** `Hunt-KQLLab-Pilot`
- **Custom detection (capstone):** `Detect-KQLLab-Pilot` (alert-only)
- **Workbook (lab):** `Workbook-KQLLab-Pilot`
- **Knowledge-check:** `Check-KQLLab-Pilot`

## What this guide deliberately does NOT do

• It does **not** deploy or reconfigure the platforms — onboarding Defender XDR, connecting Sentinel, and licensing the pillars are the **Defender XDR onboarding / Sentinel deployment / per-pillar** catalog solutions. This guide **queries** them.

• It does **not** replace the **Threat Hunting hunt-library** solution — that delivers and operates a curated hunt *program*; this builds the *skill* and ships a *learning* lab pack (adjacent, not duplicate).

• It does **not** take response actions — labs are **read-only**, the capstone detection is **alert-only**, and any real finding routes to the SOC's normal incident process.

• It does **not** decide **which detections to promote or acceptable false-positive rates** — those are security-leadership determinations; the guide provides the skill, the labs, and the data.

• It does **not** invent schema, licensing, or preview facts — where availability may differ by tenant/cloud (Sentinel, table schemas, plan-gated features) it says **verify in your tenant**.

## How it is delivered

This guide is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus the KQL lab pack and all four PowerShell helpers, in a single ZIP (`WSKQL-Complete-Solution.zip`) — yours to keep and re-run for every new cohort.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

## WSKQL-Discovery.ps1

**Purpose:** Confirms the environment can support the labs before the cohort arrives: which Defender XDR advanced-hunting tables have recent data, whether a Microsoft Sentinel / Log Analytics workspace is reachable for the cross-tool labs, and the learner cohort-group membership. Read-only — it changes nothing. HOW TO RUN pwsh -File .\WSKQL-Discovery.ps1 pwsh -File .\WSKQL-Discovery.ps1 -Cohort "Grp-KQLWorkshop-Cohort" -Environment USGov

**How to run:** pwsh -File .\WSKQL-Discovery.ps1 pwsh -File .\WSKQL-Discovery.ps1 -Cohort "Grp-KQLWorkshop-Cohort" -Environment USGov

## WSKQL-Labs.kql

**Purpose:** // The runnable, progressive KQL library for the workshop. One labeled section per lab // (fundamentals -> filtering -> aggregation -> joins -> cross-tool -> real hunt -> // detection candidate -> workbook query) plus the capstone. This is the core take-home // artifact — keep it and grow it into your team's shared query library. // HOW TO RUN // Paste a single query (one contiguous block below a "// Lab N" header) into either: // - Microsoft Defender: security.microsoft.com -> Hunting -> Advanced hunting // - Microsoft Sentinel: Sentinel -> Logs // Or run one read-only from PowerShell: WSKQL-RunLab.ps1 -Lab <n> // EXPECTED RESULTS // Each query returns a small, explainable result set. Where a table/column may differ in // your tenant/cloud it is flagged // VERIFY — correct it against Advanced hunting -> schema. // NOTES // White-label. READ-ONLY — every query only reads telemetry; nothing here changes the // tenant. Defender XDR tables use the column `Timestamp`; Sentinel tables use // `TimeGenerated`. Table/column names evolve — VERIFY current schema in your tenant. // ========== ==========

**How to run:** // Paste a single query (one contiguous block below a "// Lab N" header) into either: // - Microsoft Defender: security.microsoft.com -> Hunting -> Advanced hunting // - Microsoft Sentinel: Sentinel -> Logs // Or run one read-only from PowerShell: WSKQL-RunLab.ps1 -Lab <n> // EXPECTED RESULTS // Each query returns a small, explainable result set. Where a table/column may differ in // your tenant/cloud it is flagged // VERIFY — correct it against Advanced hunting -> schema. // NOTES // White-label. READ-ONLY — every query only reads telemetry; nothing here changes the // tenant. Defender XDR tables use the column `Timestamp`; Sentinel tables use // `TimeGenerated`. Table/column names evolve — VERIFY current schema in your tenant. // ========== ==========

## WSKQL-Report.ps1

**Purpose:** Produces a shareable summary of the workshop: for each lab, how many rows its query returns against the live tenant right now (a quick "does this lab have data to teach on?" signal), the cohort size, and a place to record knowledge-check scores. Read-only — it only queries and reports. HOW TO RUN pwsh -File .\WSKQL-Report.ps1 pwsh -File .\WSKQL-Report.ps1 -Cohort "Grp-KQLWorkshop-Cohort" -OutputCsv .\wskql-cohort.csv

**How to run:** pwsh -File .\WSKQL-Report.ps1 pwsh -File .\WSKQL-Report.ps1 -Cohort "Grp-KQLWorkshop-Cohort" -OutputCsv .\wskql-cohort.csv

## WSKQL-RunLab.ps1

**Purpose:** Lets a learner execute one lab's query from PowerShell as well as the portal, so the whole cohort can run the same query and compare results. It carries a small, self-contained copy of each lab's Defender-XDR query so it runs without parsing the .kql file. Read-only — it only queries telemetry. HOW TO RUN pwsh -File .\WSKQL-RunLab.ps1 -Lab 1 pwsh -File .\WSKQL-RunLab.ps1 -Lab 6 -LookbackDays 2 -OutputCsv .\lab6.csv

**How to run:** pwsh -File .\WSKQL-RunLab.ps1 -Lab 1 pwsh -File .\WSKQL-RunLab.ps1 -Lab 6 -LookbackDays 2 -OutputCsv .\lab6.csv

## WSKQL-Validate.ps1

**Purpose:** Turns 05-testing.md into a checklist run: confirms the environment supports the labs (tables have data), that the lab queries resolve, that the capstone detection Detect-KQLLab-Pilot exists and is ALERT-ONLY, that the saved-query set and workbook are present (manual where the API does not expose them), and scores the knowledge-check Check-KQLLab-Pilot. Read-only. HOW TO RUN pwsh -File .\WSKQL-Validate.ps1 pwsh -File .\WSKQL-Validate.ps1 -DetectionName "Detect-KQLLab-Pilot" -Environment USGov

**How to run:** pwsh -File .\WSKQL-Validate.ps1 pwsh -File .\WSKQL-Validate.ps1 -DetectionName "Detect-KQLLab-Pilot" -Environment USGov